AI-Enabled Grievance Redressal – A comprehensive technical report

Overview. This project applies data science and natural language processing (NLP) techniques to modernize Janasunani, the Government of Odisha's grievance redressal platform, by improving efficiency in handling large-scale filing of complaints between April 2021 and June 2025. We will develop workflows to digitize unstructured documents, extract entities such as schemes, departments, and locations, automatically summarize content within these filed grievances, and classify complaints into thematic categories.

The outputs of this pipeline will include a structured database of grievances with relevant metadata surrounding their filing, anonymization and summarized versions of these documents. We leverage state-of-the-art NLP solutions for each of these tasks with:

- Token-level classification techniques to extract and redact personally identifiable information from within grievance related content
- Classification of pages of grievance related documents based on the format or type of information
- Thematic classification of anonymized grievance related content
- Extractive or abstractive summarization techniques to generate informative content summaries

Beyond reducing manual entry and administrative burden, the work will generate a tool that reveals actionable insights at scale, turning citizen grievances into feedback that can be directly incorporated for governance improvement, and demonstrating how data analysis can strengthen trust between governments and citizens.

Pipeline. Our corpus consists of a large collection of petitions produced in variable formats and types. The majority of these petitions are stored in the form of PDF files consisting of a letter and one or more accompanying documents. These documents can include but are not limited to government identification, energy bills, hospital bills, death certificates, etc. A quick review of the corpus reveals that the majority of petitions are multi-page documents where each page can be one of the following format types: 1) Scanned, 2) Printed, 3) Handwritten.
In addition we also find these documents to be presented either in colour or in black & white text as well as of variable image quality. To add some structure and to better understand the document corpus, we first develop a ***Document Format Classifier*** that is able to segment documents into individual pages and extract some relevant pagewise metadata along three dimensions: 1) format, 2) language, and 3) quality . This helps us to categorize documents prior to text extraction and other downstream tasks. Following this step, documents are forwarded to a ***Text Extraction*** module enabled with Optical Character Recognition that can parse the contents of PDF or image files and extract their contents in textual format. A large proportion of our documents contain Personally Identifiable Information (PII) that needs to be anonymized prior to these contents being processed for downstream NLP goals. To this end, we develop a ***PII Tagger*** that leverages token-level classification and consists of a BERT based large language model that reads the textual contents of a document and is finetuned to tag each

token with a label that specifies its location with respect to the nearest PII within text. Once PII spans are extracted from within text, the PII within these spans are anonymized by using a pseudo-token to protect the privacy of the authors of these documents. Finally, PII anonymized content is often used for generating informative summaries for efficient record keeping as well as clustering these texts based on underlying topical trends. Figure 1. Illustrates the conceptualization of our end-to-end grievance redressal pipeline.

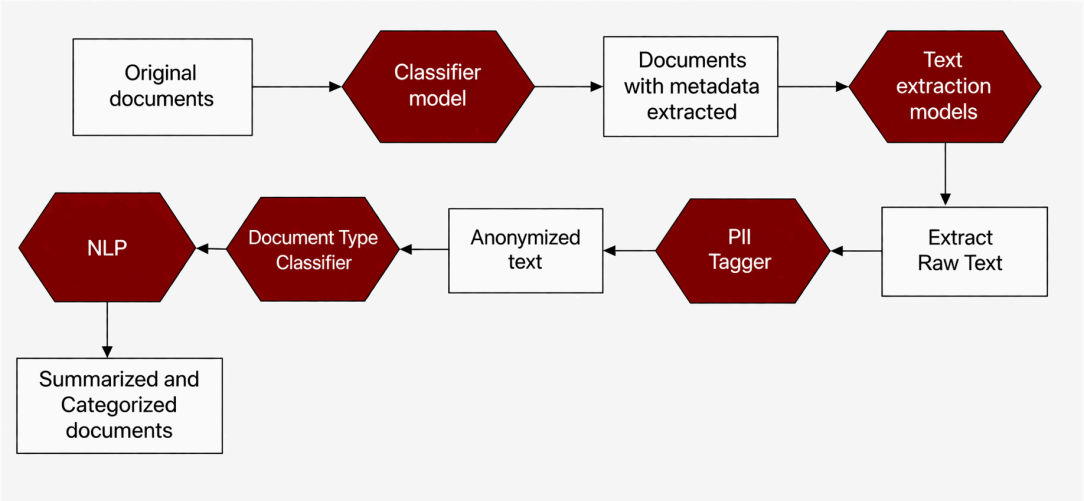


Figure 1: AI Enabled Grievance Redressal Pipeline

1. Document Format Classification. We leverage a simple OCR enabled text extraction tool named PyTesseract to extract image features and train Random Forest models that are able to process individual pages of a document and classify them along the following dimensions:

- **Format**: Scanned/Not Scanned
- **Language**: English/Odia/Both
- **Content format**: Handwritten/Printed
- **Image Quality**: High/Medium/Low

In addition to image features from PyTesseract we also augment 27 additional advanced features from:

- BRISQUE quality metrics : Image quality assessment to determine whether an image is distorted or not. evaluate the perceptual quality of an image without requiring a pristine reference image.
- GLCM texture analysis : (Gray-Level Co-Occurrence Matrix) texture analysis is a statistical method that quantifies the spatial texture features of an image.
- FFT frequency features : Quantifies the magnitude and phase of different spatial frequencies registered in an image using Fast Fourier Transform.

To deal with underlying class imbalance, we use SMOTE based class balancing and data augmentation techniques (rotation, brightness, contrast, noise). The resulting classifiers are then utilized to build a document database upon processing all documents, segment them into pages and add pagewise metadata based on the aforementioned categorizations.
Here we present a breakdown of the grievance corpus by language and format. Document Breakup displays the distribution of pages across language categories, further broken down by page type (Printed, Handwritten, Both, or No-Text/Unknown). Page-level counts are used as the

primary metric since a single document can contain a mix of page types.

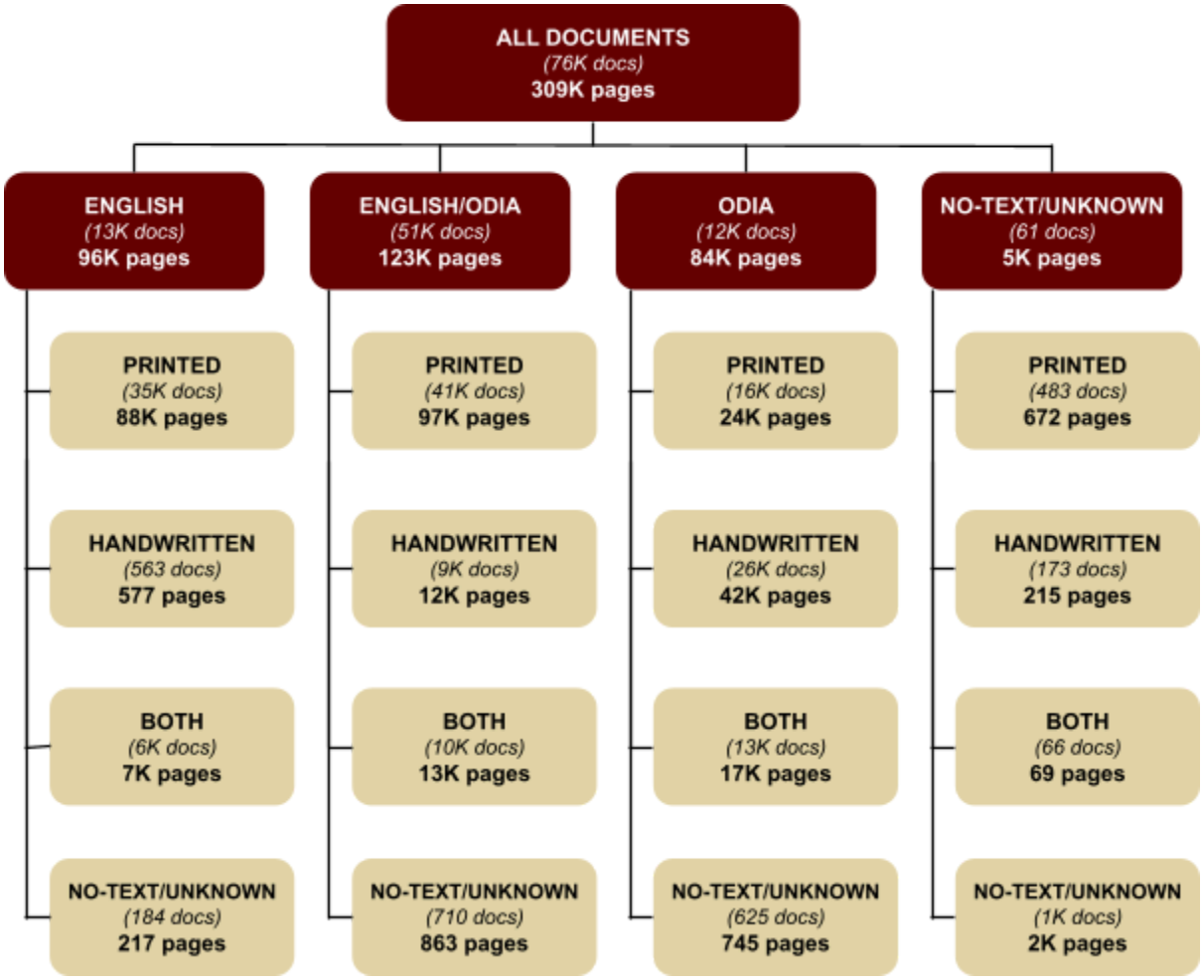


Figure 2: Breakdown of Grievance Data

Language	Total Pages	Count of Classified Text	Drop-out rate
English	96,486	96,269	0.22%
English/Odia	123,285	122,422	0.7%
Odia	83,815	83,070	0.88%
No text/unkown	5,259	3,499	66.53%

*Dropout Rate reflects proportion of pages not classified as **Printed, Handwritten, or Both.** This may be due to the page containing a picture with no text, or the classification model being unable to determine the correct category.*

Table 1: Handwritten & Printed Pages

Benchmarking: We conducted an initial performance analysis of the format classifiers on a sample of 1000 pages by manually labelling them across all the aforementioned categories. In

the following table we provide performance numbers of each classifier by obtaining a detailed report of the class-wise Precision, Recall and F1 scores.

Purpose	Models / Tools Tested	Key Metric	Result
Classifies each document page by physical characteristics: scanned vs. not scanned, handwritten, image quality, colour scale, and language. Outputs are used to route pages to the appropriate OCR method.	XGBoost+ OpenCV+ SMOTE	Average accuracy across classifiers	75.71%

Metric	Value
Best classification model -- accuracy on labelled data	81.97%
Best classification model -- macro precision	72.93%

Baseline Performance on initial sample of 1000 pages

2. OCR Enable Text Extraction. We test a variety of text extraction models and settle on using a state-of-the-art deep learning based OCR algorithm, namely DeepSeek-OCR. A pretrained checkpoint of this model is available on the HuggingFace platform that can be deployed locally to both finetune or run inference. Nonetheless, this model has significant computational overhead that we resolve by deploying the model on the DSI cluster that can support experiments with more computational resources. We also do not perform any additional fine tuning and directly use the model to run inferences and extract text from PDFs as well as generate confidence scores to estimate the reliability of text extraction. We also conduct a qualitative analysis of the quality of extraction on a small sample of documents.
In our experience, DeepSeek-OCR is capable of accurately extracting text from well formatted PDFs in English. However, since Odia is not widely supported at this point in popular off-the-shelf OCR models or known to the team, verifying extraction accuracy is difficult for documents in this language. For now our efforts on text extraction are therefore limited to PDFs in English.

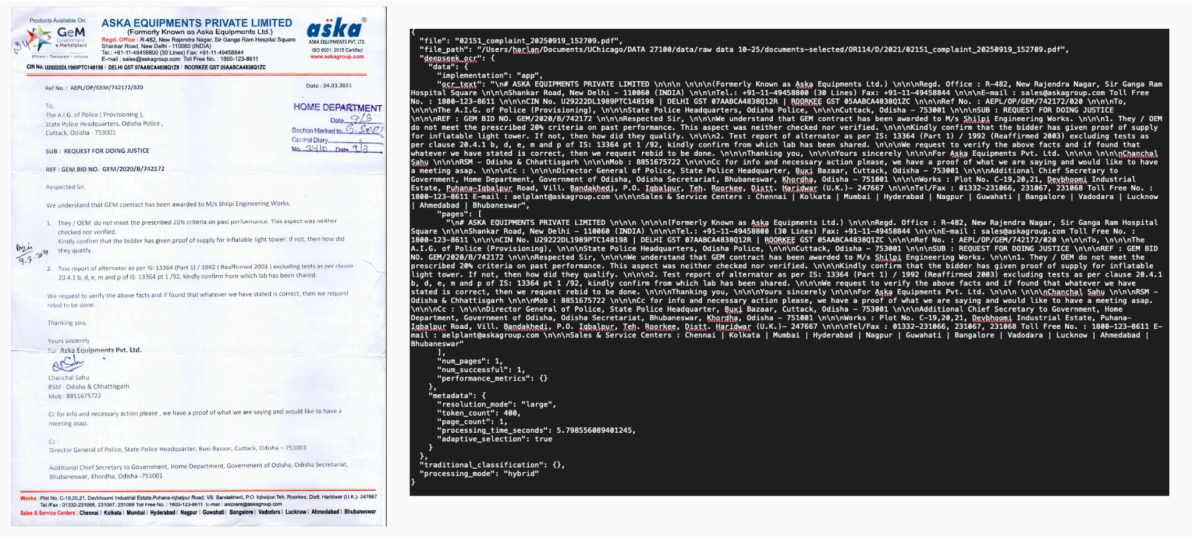


Figure 3: Sample PDF and corresponding text extraction using DeepSeek-OCR

Benchmarking:

=== OCR Quality Metrics (English-only deepseek pages) ===
Total pages: 96469

Thresholds: words >= 20, alpha_ratio >= 0.5, top_trigram_share <= 0.25

Pass word count:	82499 (85.52%)
Pass alpha ratio:	81073 (84.04%)
Pass repetition:	87925 (91.14%)
Pass ALL three:	75137 (77.89%)

- Pages where extraction_model = 'deepseek' and Language = 'english' (case- and whitespace-normalized) were evaluated ⇒ sample size: 96,469 pages.
- DeepSeek is the only OCR model whose output is identifiable in the database. There are 3 groups in extraction_model:
    - 'deepseek' — 101,238 rows.
    - NULL — ~26,775 rows with no OCR run.
    - Empty string — ~282,070 rows whose text was populated by an unknown earlier process (possibly direct PDF text extraction on born-digital docs, not OCR). These cannot be attributed to a specific OCR model and were excluded.
- Restricting to extraction_model = 'deepseek' ensures every row in the denominator was produced by the same pipeline, so the metrics describe DeepSeek's behavior rather than a mix of sources.
- No hand-transcribed ground truth exists for these pages, so standard OCR accuracy metrics (CER, WER) cannot be computed. Instead used these metrics:

    - Word count ≥ 20 — 85.52% pass
        - Flags pages where OCR returned little or no text on a presumably non-blank scan; Catches complete failures, near-empty outputs, severely truncated generations.
        - Legitimately short pages (covers, dividers, mostly-image pages) fail this even when OCR worked correctly
    - Alphabetic character ratio ≥ 0.5 — 84.04% pass.
        - Computed as alphabetic characters (Unicode-aware) divided by total characters. Flags output dominated by symbols, digits, or garbage tokens.
        - Although, numeric pages (tables, ledgers) can legitimately have low alpha ratios
    - Top trigram share ≤ 0.25 — 91.14% pass.
        - Computes the share of all trigrams accounted for by the single most frequent trigram. Flags repetition-collapse, DeepSeek's signature failure mode where it loops on the same phrase.
        - Catches degenerate generations where the model gets stuck repeating a token or phrase… could fail where pages are intended to have repetitiveness
    - Pass ALL three tests — 77.89% pass.
        - The 22.11% that fail at least one check represent the OCR backlog needing review, not necessarily that they are wrong, but instead pages where the output is implausible enough that we cannot trust it without inspection
- There were 21332 pages of “failures”, i.e. any page that fails at least one of the three checks
    - Anything with share > 0.5 should be considered erroneous and that was in line with our manual inspection.

3. PII Anonymization. Personally Identifiable Information (PII) can be used on its own or in conjunction with other information to identify and locate specific individuals. As such it is important to properly anonymize this type of information prior to passing the extracted text through the rest of the pipeline for further downstream analysis. PII anonymization is crucial to preserve privacy of relevant parties, maintain compliance with legal standards and other ethical requirements.

To this end, we develop a token-level classification model to directly follow OCR based text extraction. We first adopt Beginning-Inside-Out (BIO) encoding to manually label the spans of PII’s within text. Then we finetune a BERT based large language model (RoBERTa) that can split text into individual tokens and label them to locate PIIs within text. This model is locally trained and deployed to run inference on the output of the preceding text extraction model. Tagged PIIs will be replaced in-place with a pseudo-token to maintain anonymity of sensitive information. This helps us to retain the context in which PIIs appear in text without revealing its contents. PII Anonymization precedes all downstream NLP tasks.

Benchmarking: We curate a dataset of 21,937 tokens grouped into 1,057 sentences and annotate them using the following token-level labels: **B-PII, I-PII, O**. A 90/10 split is applied and the validation set of 106 sentences (2,126 tokens) is used to test model performance. In the following table we provide performance numbers of each classifier by obtaining a detailed report of the class-wise Precision, Recall and F1 scores for each encoding label.

	Precision	Recall	F1	Support
B-PII	0.963	0.897	0.929	29
I-PII	1.000	0.575	0.730	40
O	0.991	1.000	0.995	2057

Baseline Performance on initial sample of **1,057 sentences**

The model performs well at identifying the beginnings of PII spans (B-PII recall = 0.90) but exhibits weaker recall on continuation tokens (I-PII recall = 0.58). This suggests entity boundary completeness is the primary improvement target. This is addressable through more examples, alternative tagging schemes, or longer context windows.

1. Training Set Size/Test Set Size
2. Classification report per label
3. Coverage of PII per page as a percentage

In addition, we also use PII coverage as an additional metric to gauge the effectiveness of the anonymization process. Coverage is measured by first obtaining a count of the total number of PII entities within the content and then calculating the percentage of those entities that are detected. We obtain coverage of 80.56% on our test set (counting any overlap with a true span; 50.00% under strict exact-span matching).

While the previous performance analysis is more informative of how we can improve model training, coverage is a more interpretable and useful metric for practical purposes.

PII anonymized texts are then utilized for several downstream NLP goals that can better characterize the information stored in them. To this end, we attempt to develop solutions for the following tasks:

1. Automated Summarization to generate informative summaries of the contents of grievances.
2. Thematic Classification of Grievances : Topic modelling to understand the underlying thematic focus of complaints, for ex: Land Dispute, Taxation etc.

4. Thematic Classification of Grievances. We leverage text classification to generate thematic clusters within documents. For example, categorizing documents into topics like Land Disputes,

Taxation etc. helps to easily organize grievances into topics that can later be useful for easy search and indexing. To this end, we use a dataset of 84K summarized grievances. Some of these grievances also have supporting documents within the aforementioned corpus whose content we can add to the summary if needed. Note that, due to the choices made upstream, we only process and add documents in english as supporting documentation to grievances.

Benchmarking: We use an off-the-shelf pretrained BERT model called MuRIL with multilingual capabilities specifically designed for 17 Indian languages. The model is finetuned on a training sample of 6598 grievances and tested on 84990 grievances. We conduct some additional data cleaning and missing data checks and drop entries where the top level category level as well as the subcategory were both missing. For the ones that only had a subcategory label, we treat that as the label to predict. We put these entries in a separate smaller test set and evaluate our trained model's performance on this dataset as well. Following these steps, the final test data consists of 65999 entries. The table below presents a detailed classification report explaining the model's performance across different topic classes on this test data.

	Precision	Recall	F1-Score	Support
Education	0.6588	0.6325	0.6454	1581
Energy	0.8038	0.7591	0.7808	1457
Environment & Utilities	0.5474	0.5443	0.5458	3270
Financial Assistance	0.5058	0.5599	0.5315	2104
Health Care	0.6522	0.6007	0.6254	1202
Housing	0.7096	0.9259	0.8034	22129

Infrastructure	0.7280	0.7939	0.7595	6496
Legal	0.7285	0.7992	0.7622	3899
Police Case	0.8784	0.8248	0.8508	4372
Service Matters	0.7538	0.7012	0.7266	3441
Social Welfare	0.7118	0.3936	0.5069	16048
Accuracy			0.7104	65999
Macro Average	0.6989	0.6850	0.6853	65999
Weighted Average	0.7118	0.7140	0.6947	65999

The second NLP task is summarization of grievance related documents to produce informative gists that can present the main points of a grievance, and include key information loci, while eliminating the need to go through lengthy documents that contain a lot of different types of information. However, one thing that proves challenging in this context is the variety of supporting documentation that are submitted alongside a grievance. Off-the-shelf summarizers do not work very well when consolidating information from multiple sources ranging from letters or text to identification cards or receipts and generating a single coherent summary.

5. Page Type Classifier. To solve the aforementioned problem, we develop a page type classifier that first segments a grievance related document page by page and then classifies each page into one of the following types:

a. Identification - Counting any form of ID no matter of format
b. Letter - From a person to a person/entity
c. Photos - a photograph of anything except a document/text
d. Text Only - If pure English text
e. Bills/Receipts - List of items with an amount of money attached
f. Form/Application - where the person has to fill out specific segments
g. Unreadable/Miscellaneous

The idea is to use the page type label as a filter and only consider the page types (within a document) that are most informative towards generating coherent and informative summaries. To develop this classifier we test the following models:

a. ViT: Vision Transformer can directly process images and PDF type pages
b. Vit+BERT: Vision Transformer supplemented with a BERT model that can additionally consume text data. We feed the output of the OCR module alongside the page in its original format.
c. Vit+Longformer: Vision Transformer supplemented with a Longformer model that can additionally consume text data. We feed the output of the OCR module alongside the

page in its original format. Longformer is better than BERT when it comes to dealing with longer texts.

Benchmarking. To finetune and evaluate each model, we generated a random sample of 1500 pages and manually labelled them based on the aforementioned types. We then conduct a 70/30 split of training to evaluation ratio where the training data is used for finetuning. Here we provide both training and evaluation performance of the three models that we consider. On the whole it seems that using an image based model in conjunction with a text based one produces the best results.

Test Set

General Performance on Test Set

Model	Accuracy	Balanced Accuracy	Macro Accuracy	Macro F1	Macro Recall	Macro Precision
ViT	0.67	0.63	0.89	0.62	0.62	0.63
Vit+BERT	0.64	0.62	0.89	0.6	0.61	0.65
Vit+Longformer	0.65	0.60	0.88	0.60	0.61	0.60

F1 Score for Test

Class	ViT	ViT+BERT	ViT + LongFormer	n
Bills/Receipts	0.51	0.47	0.53	30
Form/Application	0.59	0.53	0.49	45
Identification	0.70	0.64	0.67	26
Letter	0.79	0.76	0.78	103
Misc/Not Sure	0.44	0.47	0.44	42
Text Only	0.70	0.74	0.71	82

Training Set

General Performance for Train

Model	Accuracy	Balanced Accuracy	Macro Accuracy	Macro F1	Macro Recall	Macro Precision
ViT	0.997	0.997	0.997	0.997	0.997	0.997

Vit+BERT	0.9	0.83	0.97	0.85	0.83	0.9
Vit+Longformer	0.997	0.997	0.997	0.997	0.997	0.997

F1 Score for Train

Class	ViT	ViT+BERT	ViT + LongFormer	n
Bills/Receipts	0.51	0.62	0.53	30
Form/Application	0.59	0.84	0.49	45
Identification	0.70	0.89	0.67	26
Letter	0.89	0.98	0.78	103
Misc/Not Sure	0.44	0.83	0.44	42
Text Only	0.70	0.93	0.71	82

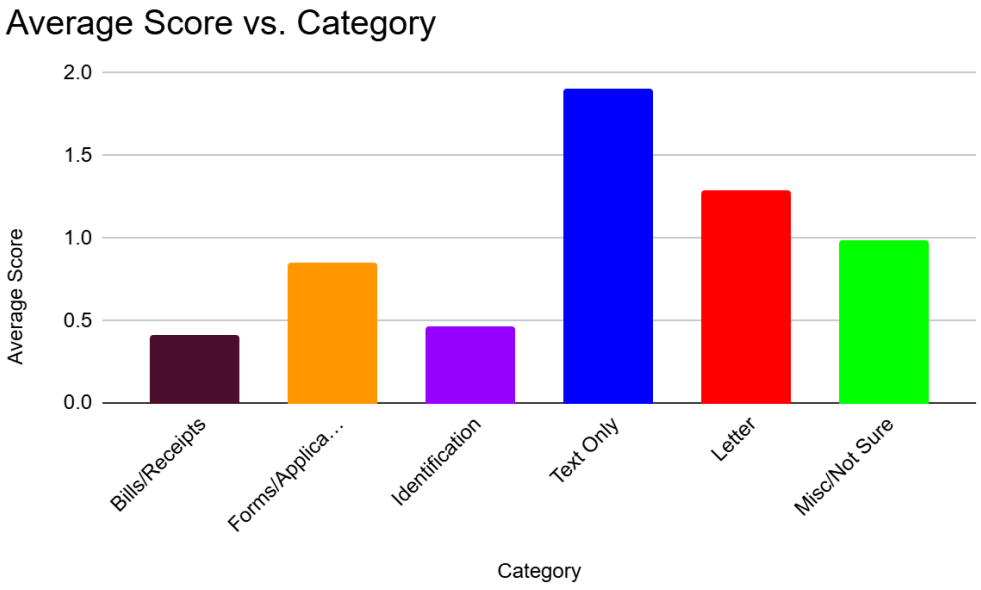
6. Grievance Summarization. The final module of the pipeline focuses on grievance document summarization. As mentioned above, we use the Page Type Classifier as an additional filtering step to weed out some pages from a grievance document that are deemed to be less informative and more prone to adding noise to the summaries.
To determine which page types are most appropriate for summarization, we perform a qualitative analysis of the quality of summaries generated for a randomly selected sample of 500 pages.

Benchmarking. **campm@uchicago.edu**

a. (Rouge_Score_CSV): This link contains the Rouge Score results for all of the different page types. The three different metrics we use for evaluation are 1) ROUGE-1 which measures the overlap of individual (unigrams) words between the summary and the original, 2) ROUGE-2 does the same but with two-word pairs (bigrams) and for this reason the scores are typically lower, 3) ROUGE-L looks at maintaining flow and sentence structure. The precision of all of these scores indicates what proportion of the original's content is captured in the summary, and the recall indicates what proportion of the original content is captured in the summary. F1 is a balance of both. Overall, while ROUGE remains a standardized metric to benchmark different models, it was not a very accurate indicator of summary quality here. Hence a more subjective, qualitative analysis is conducted on the generated summaries.

b. Link the qualitative analysis: (Overall Spreadsheet) (Qualitive_Analysis_Overall) (Qualitative_Analysis_Summarized)
As stated above, the qualitative analysis was conducted by a reviewer individually reading all 500 pages and the LLM generated summaries and then assigning a score within the range 0-3 based on how useful or not useful a document was. A score of 0 was given to any summary I found to be not useful at all, 1 was given to documents that had somewhat useful summaries but were incomplete, 2 was given to pretty useful summaries that were almost ideal. And 3 was given to summaries with useful information. Any average score near or above 1 makes the summaries useful and practical, and the only two that can be disregarded altogether are Bills/Receipts and Identification, which would seem logical regardless of the scores. Results of this experiment can be found in the documents liked above.

c. Our results here are along expected lines. Text-only pages and letters are the most useful page types when used to generate coherent summaries of grievance documents. Forms/Applications are also somewhat relevant while Bills/Receipts are deemed least usable.



Pipeline Implementation: An end-to-end implementation of this pipeline can be found here. The pipeline was implemented in Python 3.10.12 on an NVIDIA A100 80GB PCIe GPU (81920 MiB VRAM, driver 595.71.05) on a Linux cluster (kernel 5.15.0, Ubuntu).
The following diagram illustrates the workflow of the pipeline. A sample input document is fed to the pipeline in a PDF or JPEG format. Some modules within the pipeline operate on a page level while the others consider the content of the entire document as a whole and generate document-level outputs. The distinction is made clear in the diagram below.

Document Processing Pipeline

PDF + JPG ingestion → page-level table → document-level table

Input
PDF + JPG files (page level)

★ *Pre-step: if no DB exists, create one*

↓

1. format_classifier
Fills SQL DB → produces table (a) with metadata
Allows filtering by handwritten / language
writes column → master table state: (a)

↓

2. ocr_extraction
Filter table → extract text
Manual choice of engine: Pytesseract or DeepSeek (2 types)
writes column → master table state: (b)

↓

3. PII_tagger
Adds redacted_text column
writes column → master table state: (c)

↓

4. page_type_classifier
Adds page_type column
writes column → master table state: (d)

★ SPLIT POINT — page level → document level

↓

5. summarizer
Creates new table at document level (+ document_id column)
Filter by 3 data types
Adds summary column
creates new table (e), then writes column → state: (f)

↓

6. categorizer
Adds grievance, grievance_doc, and category columns
writes columns → master table state: (g)
(three columns added in this step)

↓

Output: 2 tables

Table 1 — Page level	Table 2 — Document level
Result of steps 1 – 4	Result of steps 5 – 6
One row per page	One row per document

Note on the (a) – (g) labels

The letters (a) through (g) track successive states of the master table as each step modifies it. Each letter marks a change — a new column added or, at the split point, a new table created. They are not separate deliverables; they are version markers showing what the table looks like after that step has run.

Efficiency Analysis: In order to estimate how efficiently the pipeline processes documents we devise an experiment that records the following:

- how much time does it take to run a document end to end
- a breakdown of running time per pipeline module for a fixed number (per 500 tokens) of tokens

To this end, we process several documents end-to-end through the pipeline and record the following run times (in seconds):

Document		**Run Times (per model)**					
Ticket No.	*Token Count*	*Format Classifier*	*OCR*	*PII-Tagger*	*Page Type Classifier*	*Summarizer*	*Categorizer*
CMO2023202145	838	9s	1m 31s	28.8s	7.1s	7.6s	10.2s
CMO2023221888	3,751	23.8s	1m	7.2s	7.3s	7.1s	11.5s
CMOFF/E/2021/06248	8,871	1m 40s	3m 20s	7.1s	39.1s	6.5s	14.5s
CMO20241024797	4,312	29.2s	1m 11s	8.8s	9.1s	6.8s	9.1s
CS2023226374	904	9.2s	22.6s	6.8s	5.3s	4.6s	8.3s

Upon executing several documents through the pipeline, we estimate the run time for each module per 500 tokens. We then average out these runtimes collected across all documents to determine the estimated run-time of each module of the pipeline per 500 tokens. These can be listed as follows:

- Format Classifier: 4.53 seconds
- OCR Extraction: 18.86 seconds
- PII Anonymizer: 4.67 seconds
- Page-Type Classifier: 2.49 seconds
- Document Summarizer: 1.84 seconds
- Document Categorizer: 2.82 seconds

Cost Analysis: How much cost would these tasks individually incur on AWS?
We could repeat processing all the documents used in the previous experiment to note the processing cost in terms of the number of tokens consumed and generated per example. Additionally processing times can also be compared between the proprietary models and our open source ones to get a more well-rounded efficiency analysis.
However, an accurate estimation of these factors is out of scope for the clinic due to lack of access to proprietary models.